



Bahria University, Islamabad

Department of Software Engineering

**Web Engineering Lab (Spring-
2026)**

Teacher: Engr. Subas Bilal

Student : Talha Naveed

Enrollment : 01-131232-089

Lab Journal: 07

Date: 18th March, 2026

Task No:	Task Wise Marks		Documentation Marks		Total Marks (20)
	Assigned	Obtained	Assigned	Obtained	
1	3		5		
2	3				
3	3				
4	3				
5	3				

Comments:

Signature

Lab No: 07 - React Conditional Rendering and Component Lifecycle

Objective

By the end of this lab, students will be able to: - Implement conditional rendering in React components - Understand the lifecycle of functional components using hooks - Use *useEffect* to mimic mounting and unmounting behavior - Toggle components on and off based on state changes

Tools Used

- VS Code

Link to Repository (Source Code): <https://github.com/TALHA-089/Web-Lab-07-Final.git>

Task 1: ToggleMessage.jsx

Code

```
src > ToggleMessage.jsx > default
1  import React, { useState, useEffect } from 'react';
2  import ChildMessage from './ChildMessage';
3
4  const ToggleMessage = () => {
5    // Scalable state structure for multiple independent messages
6    const [toggles, setToggles] = useState({
7      msg1: false,
8      msg2: false
9    });
10
11    useEffect(() => {
12      console.log("ToggleMessage Component Mounted");
13      return () => console.log("ToggleMessage Component Unmounted");
14    }, []);
15
16    const handleToggle = (key) => {
17      setToggles((prev) => ({
18        ...prev,
19        [key]: !prev[key]
20      }));
21    };
22
23    return (
24      <div style={{ border: '1px solid #ccc', padding: '15px', marginBottom: '20px', borderRadius: '8px' }}>
25        <h3>Toggle Messages</h3>
26
27        <div style={{ marginBottom: '15px' }}>
28          <button onClick={() => handleToggle('msg1')} style={styles.button}>
29            {toggles.msg1 ? 'Hide' : 'Show'} Message 1
30          </button>
31          /* Renders component only if condition is true, passing message as a prop */
32          {toggles.msg1 && <ChildMessage id="1" message="Hello! This is the first toggleable message." />}
33        </div>
34
35        <div>
36          <button onClick={() => handleToggle('msg2')} style={styles.button}>
37            {toggles.msg2 ? 'Hide' : 'Show'} Message 2
38          </button>
39          {toggles.msg2 && <ChildMessage id="2" message="React lifecycle concepts are powerful!" />}
40        </div>
41      </div>
42    );
43  };
44
45  const styles = {
46    button: { padding: '8px 12px', cursor: 'pointer' }
47  };
48
49  export default ToggleMessage;
```

Figure 1. Code for Component ToggleMessage.jsx

Screenshot

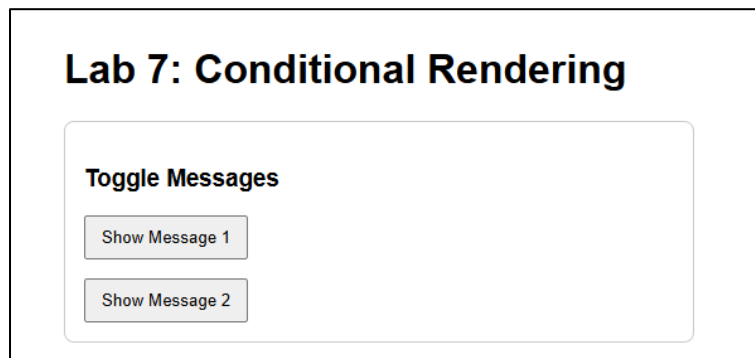


Figure 2. Messages Hidden until Toggled

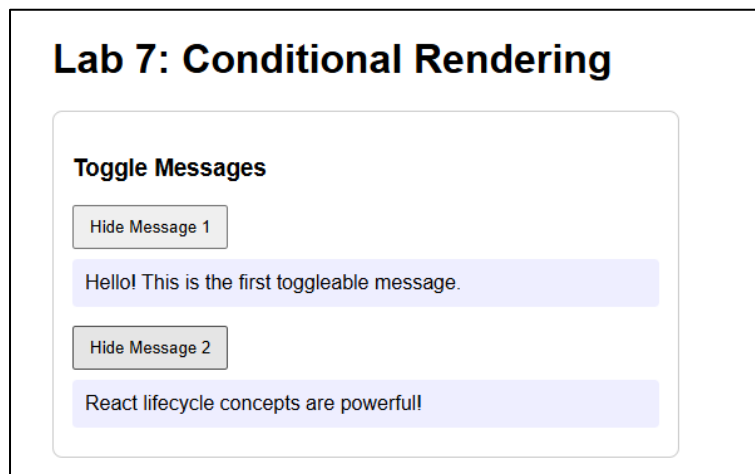


Figure 3. Messages on Display

Task 2: Adding Console logs

Screenshot

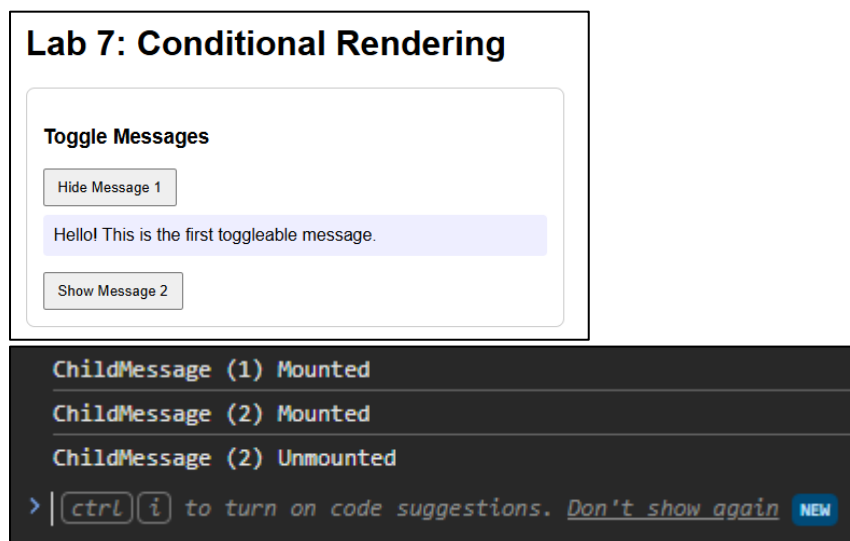


Figure 4. Console Logs

Task 3: ToggleMessage.jsx & ChildMessage.jsx

To demonstrate conditional rendering with props, I created a parent component (*ToggleMessage*) that controls a *boolean* state, and a reusable child component (*ChildMessage*) that accepts a message prop. I used the logical AND (&&) operator to conditionally render the child.

Code

```
4  const ToggleMessage = () => {
5    // Scalable state structure for multiple independent messages
6    const [toggles, setToggles] = useState({
7      msg1: false,
8      msg2: false
9    });
10
11    useEffect(() => {
12      console.log("ToggleMessage Component Mounted");
13      return () => console.log("ToggleMessage Component Unmounted");
14    }, []);
15
16    const handleToggle = (key) => {
17      setToggles((prev) => ({
18        ...prev,
19        [key]: !prev[key]
20      }));
21    };

```

Figure 5. The State Controlling the Toggles

```
32  {toggles.msg1 && <ChildMessage id="1" message="Hello! This is the first toggleable message." />}
33  </div>
34
35  <div>
36    <button onClick={() => handleToggle('msg2')} style={styles.button}>
37      {toggles.msg2 ? 'Hide' : 'Show'} Message 2
38    </button>
39    {toggles.msg2 && <ChildMessage id="2" message="React lifecycle concepts are powerful!" />}

```

Figure 6. The conditional rendering (&&) and passing the prop (message="...")

```
3  const ChildMessage = ({ message, id }) => {
4    useEffect(() => {
5      // Logs when the component is added to the DOM
6      console.log(`ChildMessage (${id}) Mounted`);
7
8      // Cleanup function runs before unmounting
9      return () => {
10       console.log(`ChildMessage (${id}) Unmounted`);
11     };
12   }, [id]); // Empty or static dependency array ensures this only runs on mount/unmount
13

```

Figure 7. Receiving the "message" prop

Screenshot



Figure 8. When toggle is OFF, the button says "Show Message 1" and there is no message below it

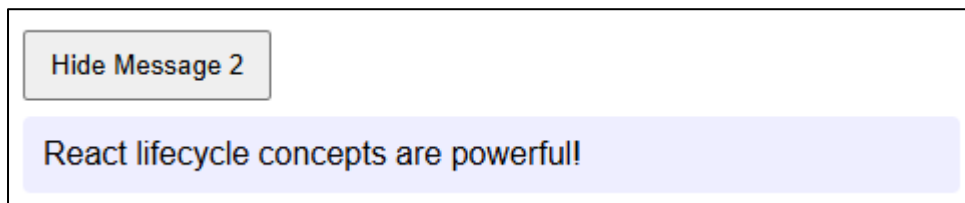


Figure 9. When toggle is ON, the button says "Hide Message 1" and the ChildMessage text is clearly visible

Task 4: Countdown.jsx

To demonstrate component lifecycle and side effects, I created a Countdown component. I used the **useEffect** hook with an empty dependency array to start a **setInterval** timer when the component mounts. To prevent memory leaks, I returned a cleanup function containing **clearInterval(timerId)**, which executes exactly when the component is unmounted from the DOM.

Code

```
1 import React, { useState, useEffect } from 'react';
2
3 const Countdown = () => {
4   const [timeLeft, setTimeLeft] = useState(10);
5
6   useEffect(() => {
7     console.log("Countdown Component Mounted");
8
9     // Performs side effect (timer) on mount
10    const timerId = setInterval(() => {
11      setTimeLeft((prevTime) => {
12        if (prevTime <= 1) {
13          clearInterval(timerId); // Stop at 0
14          return 0;
15        }
16        return prevTime - 1; // Update UI every second
17      });
18    }, 1000);
19
20    // Clears the timer on unmount using cleanup function
21    return () => {
22      console.log("Countdown Component Unmounted (Interval Cleared)");
23      clearInterval(timerId);
24    };
25  }, []);
26
27   return (
28     <div style={{ padding: '15px', border: '2px solid #ff4757', borderRadius: '8px', textAlign: 'center' }}>
29       <h2>Countdown: {timeLeft}s</h2>
30       {timeLeft === 0 && <p style={{ color: '#ff4757', fontWeight: 'bold' }}>Timer Finished!</p>}
31     </div>
32   );
33 };
34
35 export default Countdown;
```

Figure 10. Code for Countdown.jsx

Screenshot

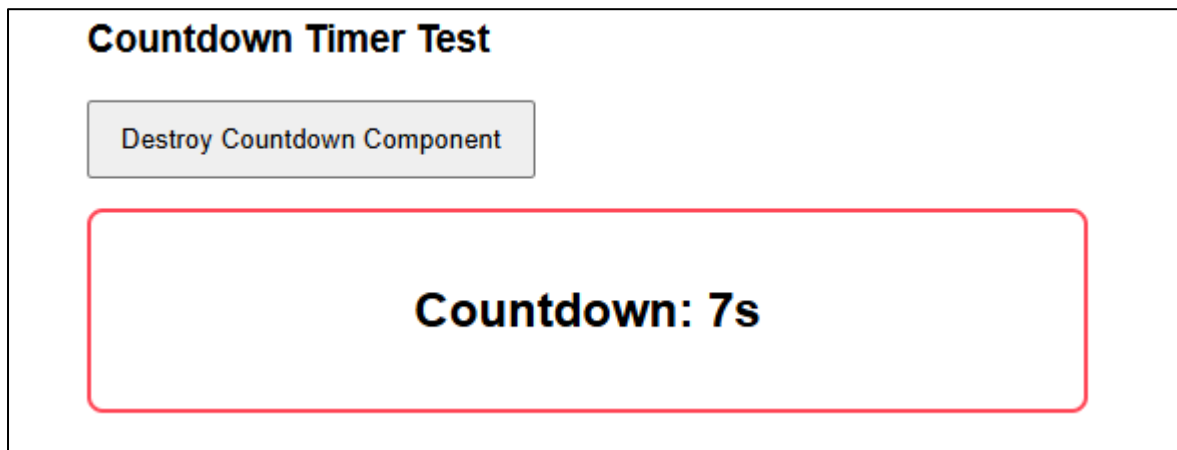


Figure 11. Countdown Component Mounted and Running (UI)

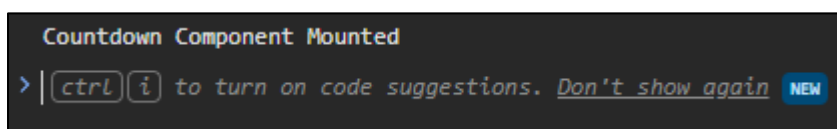


Figure 12. Countdown Component Mounted and Running (Console)

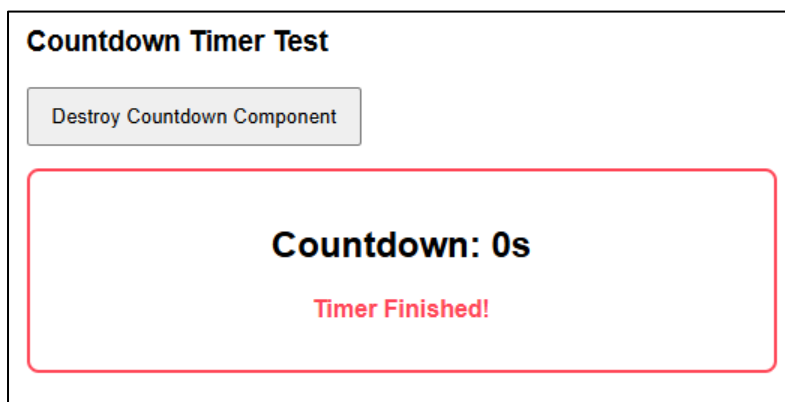


Figure 13. Countdown finished

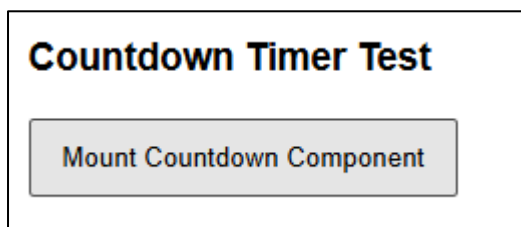


Figure 14. After Clicking the Destroy Countdown (UI)

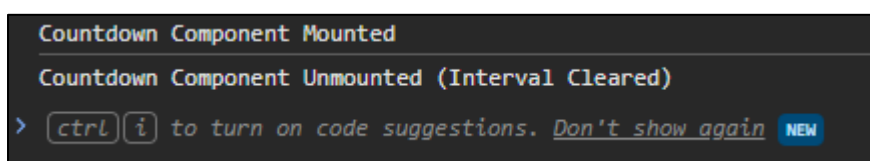


Figure 15. After Clicking the Destroy Countdown (Console)